

## **WEEK 4 – MICROSERVICES**

### **Authentication and Authorization using JWT in ASP.NET Core Web API**

#### **1. Introduction**

Authentication and Authorization are essential security features in modern web applications. Authentication verifies the identity of the user, while Authorization determines what resources the authenticated user can access.

JSON Web Token (JWT) is a secure, compact, and self-contained method used to transmit user identity information between the client and server. JWT is widely used in ASP.NET Core Web APIs to protect endpoints and provide secure access to resources.

In this hands-on exercise, an ASP.NET Core Web API was developed that authenticates users using JWT tokens, secures endpoints using authorization, and implements role-based access control.

#### **2. Objective**

The objectives of this hands-on exercise are:

- Create an ASP.NET Core Web API.
- Implement JWT Authentication.
- Generate JWT tokens after successful login.
- Protect API endpoints using JWT.
- Implement Role-Based Authorization.
- Handle expired or invalid JWT tokens.
- Test APIs using Swagger UI.

#### **3. Problem Statement**

Develop a secure ASP.NET Core Web API that allows users to log in using valid credentials.

After successful authentication:

- Generate a JWT Token.
- Return the token to the client.
- Allow authenticated users to access protected APIs.
- Restrict Admin APIs to users having the Admin role.
- Validate JWT tokens before allowing access to secured resources.

## 4. Software Requirements

| Software           | Version  |
|--------------------|----------|
| Visual Studio Code | Latest   |
| .NET SDK           | 8.0      |
| ASP.NET Core       | 8        |
| Swagger UI/        | Built-in |
| Windows            | 10 / 11  |

## 5. Technologies Used

- ASP.NET Core Web API
- C#
- JWT (JSON Web Token)
- Swagger / OpenAPI
- REST APIs
- Authorization Middleware
- Authentication Middleware

## 6. JWT Authentication Overview

JWT (JSON Web Token) is an industry-standard method used to securely transmit information between two parties.

A JWT consists of three parts:

Header

Contains:

- Token Type
- Signing Algorithm

Example:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

### Payload

Contains claims such as:

- Username
- Role
- Expiration Time

Example

```
{  
  "name": "admin",  
  "role": "Admin"  
}
```

### **Signature**

The signature verifies that the token has not been modified.

The complete JWT format is:

Header.Payload. Signature

## **8. Implementation**

### **Exercise 1 – JWT Authentication**

Implemented:

- Login API
- Username and Password validation
- JWT Token generation

Credentials used:

Username: admin

Password: admin123

After successful authentication, the application generates a JWT token and returns it to the client.

### **Exercise 2 – Secure API Endpoint**

Implemented a protected endpoint using: [Authorize]

Endpoint: GET /api/Secure/data

Only authenticated users can access this endpoint using a valid JWT token.

### **Exercise 3 – Role-Based Authorization**

Implemented Admin role support.

Added: ClaimTypes.Role = "Admin"

Created: GET /api/Admin/dashboard

Protected using: [Authorize(Roles="Admin")]

Only Admin users can access this endpoint.

#### **Exercise 4 – JWT Token Validation**

Configured JWT Bearer Authentication.

Implemented:

- Token validation
- Expired token handling
- Unauthorized access handling

If an expired token is used, the API detects it and returns an appropriate response.

### **9. API Endpoints**

#### **Login API**

Method

POST : /api/Auth/login

Purpose: Authenticates the user and generates a JWT token.

#### **Secure API**

Method

GET: /api/Secure/data

Purpose: Returns protected data for authenticated users.

#### **Admin API**

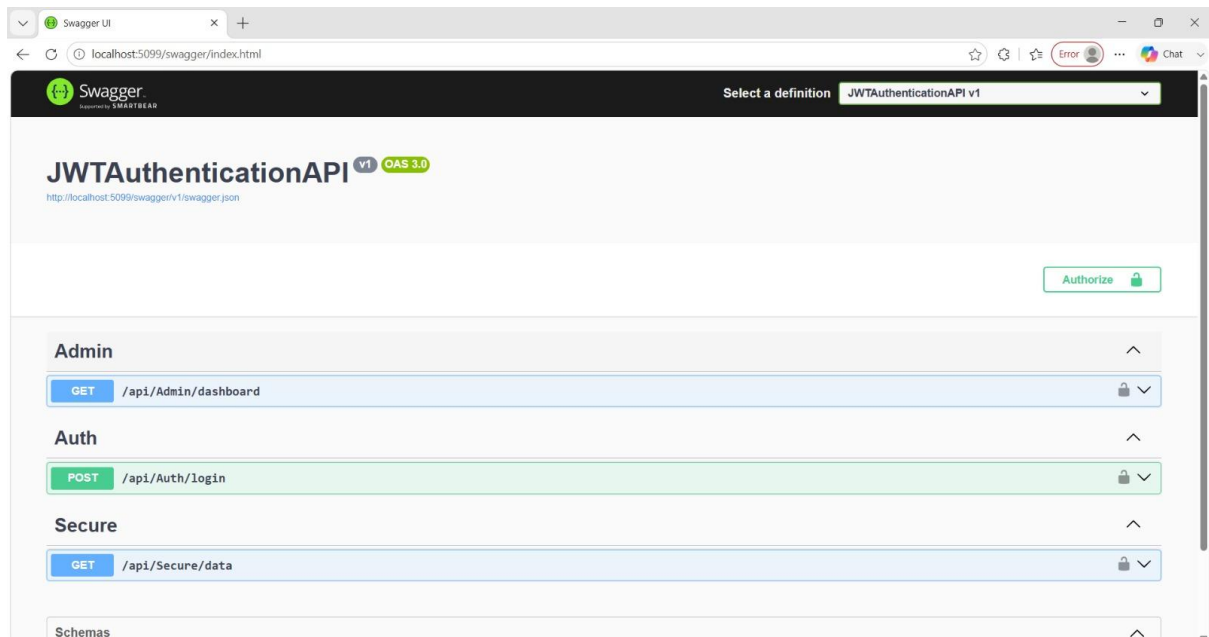
Method

GET: /api/Admin/dashboard

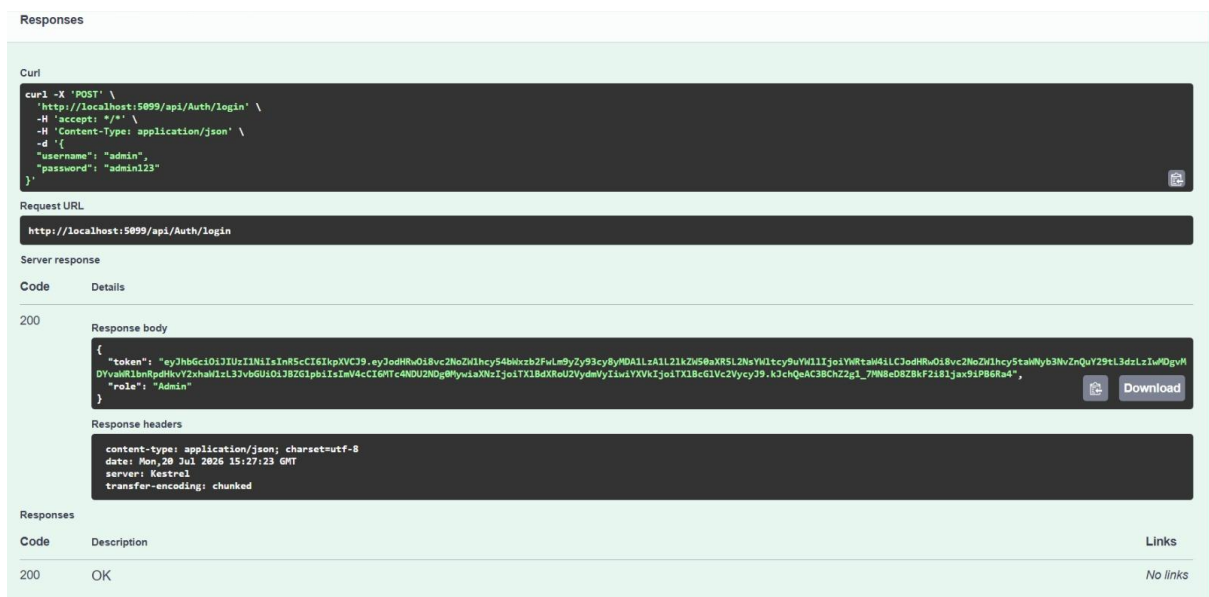
Purpose: Returns the Admin Dashboard only for Admin users.

## Swagger Home Page

## Swagger Home Page



### Login API Response showing generated JWT Token





## 12. Learning Outcomes

After completing this hands-on exercise, I learned:

- JWT Authentication in ASP.NET Core.
- JWT Token generation.
- Authentication middleware configuration.
- Authorization middleware.
- Role-Based Authorization.
- Secure REST API development.
- Testing APIs using Swagger.
- Protecting endpoints using JWT Bearer Authentication.

## 13. Conclusion

This hands-on exercise successfully demonstrated the implementation of **JWT Authentication and Authorization** in an **ASP.NET Core Web API**. A secure login system was developed to authenticate users and generate JWT tokens. Protected endpoints were implemented using the `[Authorize]` attribute, and role-based authorization was configured to restrict access to administrative resources. The application was tested using **Swagger UI**, confirming that authentication, authorization, and token validation functioned correctly. This project provided practical experience in developing secure RESTful APIs using industry-standard authentication mechanisms.